

LAPORAN PRAKTIKUM
PRAKTIKUM PEMROGRAMAN WEBSITE 1
PERTEMUAN 10
“JAVASCRIPT 1”



Disusun Oleh:

Nama	: Misbahul Huda
NIM	: 25/568209/SV/27391
Kelas	: PL2B2
Dosen Pengampu	: Achmad Choirudin Emcha, S.Kom., M.Eng.

PROGRAM STUDI D IV TEKNOLOGI REKAYASA PERANGKAT
LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

PERTEMUAN 10

JAVASCRIPT 1

1. Tujuan

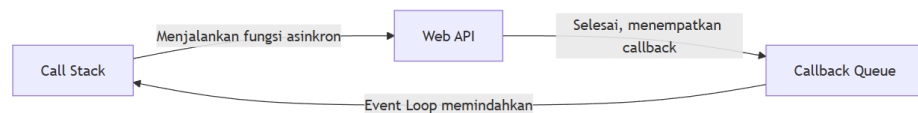
- 1.1. Mahasiswa mampu memahami dan membedakan tiga cara penyisipan kode JavaScript ke dalam halaman HTML, yaitu inline, embed, dan eksternal.
- 1.2. Mahasiswa mampu memahami dan menggunakan berbagai metode output dalam JavaScript, meliputi `console.log()`, `alert()`, `document.write()`, dan `innerHTML` untuk menampilkan data pada halaman web.
- 1.3. Mahasiswa mampu mengimplementasikan kode HTML dan JavaScript secara sederhana sebagai dasar pengembangan halaman web yang interaktif.

2. Dasar Teori

2.1. Pengertian Umum

JavaScript adalah bahasa pemrograman berorientasi objek dan fungsional dengan sintaks mirip C dan Java [1]. Dirancang agar mudah dipelajari, JavaScript awalnya berfungsi sebagai *skrip sisi-klien* yang ringan [2]. Bahasa ini bersifat *multi-paradigma*, mendukung pemrograman prosedural, objek, maupun fungsional [1]. Dalam model objeknya, setiap objek dapat mewarisi sifat dari *prototype* lain [1]. JavaScript bekerja dengan model *single-threaded* sehingga operasi asinkron ditangani menggunakan *event loop* [1]. Seiring berkembangnya teknologi web, JavaScript kini digunakan secara luas pada browser, aplikasi mobile, desktop, hingga server melalui Node.js

[2]. JavaScript bekerja dengan model *single-threaded* secara alami, sehingga operasi asinkron ditangani oleh *event loop*, yang mengantri tugas-tugas untuk dieksekusi secara berurutan. Diagram berikut mengilustrasikan alur kerja *event loop* JavaScript. Contoh ilustrasinya sebagai berikut:



Gambar 2.1 Ilustrasi sederhana mengenai alur kerja event loop pada javascript

2.2. Implementasi

JavaScript diintegrasikan dalam halaman web melalui elemen HTML `<script>`. Dengan bantuan DOM (*Document Object Model*), JavaScript dapat mengakses dan memodifikasi struktur halaman web [3]. DOM merupakan representasi pohon dari dokumen HTML yang memungkinkan setiap elemen dimanipulasi menggunakan JavaScript [3]. JavaScript juga mendukung *event handling* atau penanganan peristiwa, misalnya klik tombol, pengisian formulir, maupun gerakan mouse [3]. Ketika suatu *event* terjadi, fungsi tertentu akan dijalankan sebagai respons terhadap interaksi pengguna. Hal ini membuat halaman web menjadi lebih interaktif dan dinamis. Selain itu, JavaScript modern mendukung komunikasi data secara asinkron menggunakan Fetch API [4]. Teknologi ini memungkinkan pengambilan data dari server tanpa perlu memuat ulang halaman web sehingga meningkatkan efisiensi dan pengalaman pengguna. Dalam pengembangan modern, JavaScript juga mendukung modularisasi kode

melalui sistem `import` dan `export`. JavaScript dapat dijalankan baik di browser maupun di lingkungan server seperti Node.js [7]. Untuk meningkatkan keamanan aplikasi web, pengembang biasanya menerapkan mekanisme *Content Security Policy* (CSP) guna mencegah serangan *Cross-Site Scripting* (XSS) [5]. Selain itu, optimasi performa dilakukan dengan meminimalkan ukuran file JavaScript dan menggunakan atribut `async` atau `defer` pada tag `<script>` [6].

Contoh kodenya sebagai berikut:

```
<p id="demo">Halo</p>

<script>
document.getElementById("demo").textContent = "Halo, JavaS-
cript!";
</script>
```

Kode di atas digunakan untuk mengubah isi teks pada elemen HTML dengan `id="demo"`. Awalnya teks yang tampil adalah “Halo”, kemudian JavaScript menggantinya menjadi “Halo, JavaScript!”. Contoh ini menunjukkan bagaimana JavaScript dapat memanipulasi elemen HTML secara langsung melalui DOM [3].

3. Hasil & Pembahasan

3.1. Tugas

- 3.1.1. Silahkan buat 1 file html yang isinya memanggil code javascript yang berasal dari embed, inline, eksternal. (Isi teks html dan javascript bisa anda variasikan sendiri).

Jawab:

Pemrograman akan dibuat dalam 1 file yang dimana ada 2 file pemrograman, yaitu index.html dan script.js yang dimana keduanya saling berhubungan untuk menciptakan website yang bekerja dengan baik.

(Index.html)

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Tiga Cara KERENNNNN Memanggil JavaS-
cript</title>

  <script src="script.js"></script>

  <script>
    function tampilEmbed() {
      document.getElementById("output-
embed").textContent = "Halo dari JavaScript Embed!";
    }
  </script>
</head>
<body>

  <h1>Tiga Cara KERENNNNN Memanggil JavaScript</h1>
  <hr>

  <h2>1. Inline</h2>
  <button onclick="alert('shalom goyim')">Klik Saya
(Inline)</button>

  <h2>2. Embed</h2>
  <button onclick="tampilEmbed()">Klik Saya (Em-
bed)</button>
  <p id="output-embed"></p>

  <h2>3. Eksternal</h2>
  <button onclick="tampilEksternal()">Klik Saya (Ek-
sternal)</button>
```

```
<p id="output-eksternal"></p>

</body>
</html>

<!-- aku gatau cara kerjanya jujur tapi ini keren
banget -->
```

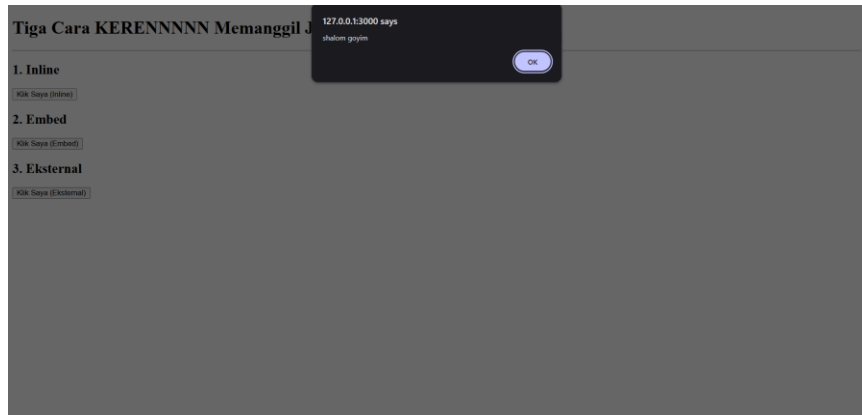
(script.js)

```
function tampilEksternal() {
    document.getElementById("output-
eksternal").textContent = "Hai helloooo :3!";
}
```

Dengan outputnya sebagai berikut:



Gambar 3.1 Tampilan output awal codingannya



Gambar 3.2 Tampilan output dari inline



Gambar 3.3 Tampilan output dari Embed



Gambar 3.4 Tampilan output dari Eskternal

JavaScript dapat disisipkan ke dalam halaman HTML dengan tiga cara berbeda, yaitu inline, embed, dan eksternal. Ketiga cara ini memiliki fungsi yang sama menjalankan kode JavaScript namun berbeda dari sisi penempatan kode, keterbacaan, dan kemudahan perawatan.

JavaScript inline adalah cara paling sederhana untuk menyisipkan JavaScript, yaitu langsung menuliskan kode JavaScript di dalam atribut HTML pada sebuah elemen. Pada contoh di file index.html, cara ini digunakan pada elemen `<button>` dengan atribut `onclick="alert('Halo dari JavaScript Inline!')"`. Artinya, ketika tombol tersebut diklik, browser langsung membaca dan menjalankan kode JavaScript yang ada di dalam tanda kutip pada atribut tersebut, tanpa perlu mencari fungsi di tempat lain.

Kelebihan cara inline adalah sangat cepat ditulis dan mudah dipahami untuk kode yang sangat pendek. Namun kekurangannya cukup besar: jika kodenya panjang atau kompleks, atribut HTML akan menjadi berantakan dan sangat sulit dibaca. Selain itu, jika tombol yang sama ada di banyak tempat, kode harus ditulis berulang kali sehingga sulit dikelola.

JavaScript embed berarti menulis kode JavaScript di dalam tag `<script>...</script>` yang ditempatkan langsung di dalam file HTML itu sendiri, biasanya di bagian `<head>` atau sebelum penutup `</body>`. Pada contoh file index.html, fungsi `tampilEmbed()` ditulis di dalam blok `<script>` di bagian `<head>`. Ketika tombol diklik, browser memanggil fungsi tersebut yang kemudian mengubah teks pada elemen `<p id="output-embed">`.

Dibandingkan dengan inline, cara embed jauh lebih rapi karena semua logika JavaScript dikumpulkan dalam satu blok, terpisah dari struktur HTML. Kelebihan lainnya adalah satu fungsi bisa

dipanggil berkali-kali dari berbagai elemen HTML yang berbeda. Namun kode JavaScript dan HTML masih berada dalam satu file yang sama, sehingga untuk proyek besar, file HTML akan menjadi sangat panjang dan sulit dikelola.

JavaScript eksternal adalah cara yang paling direkomendasikan untuk proyek yang serius. Kode JavaScript ditulis secara terpisah di file dengan ekstensi .js pada contoh ini adalah file script.js lalu file tersebut dihubungkan ke HTML menggunakan tag `<script src="script.js"></script>` di dalam bagian `<head>`. Di dalam file script.js, terdapat fungsi `tampilEksternal()` yang akan dipanggil ketika tombol ditekan.

Keuntungan utama cara eksternal adalah file HTML menjadi bersih dan hanya berisi struktur konten, sementara file JavaScript hanya berisi logika. Selain itu, satu file .js yang sama bisa digunakan oleh banyak halaman HTML sekaligus tanpa perlu menulis ulang kode yang sama di setiap halaman. Browser juga dapat men-cache file .js eksternal sehingga halaman web bisa dimuat lebih cepat pada kunjungan berikutnya. Satu hal yang perlu diperhatikan adalah kedua file harus berada dalam folder yang sama, atau path pada atribut src harus disesuaikan dengan lokasi file .js tersebut.

Secara keseluruhan, inline cocok untuk kode yang benar-benar singkat dan sekali pakai, embed cocok untuk halaman tunggal yang butuh sedikit logika JavaScript, dan eksternal adalah pilihan terbaik untuk proyek yang lebih besar dan terstruktur. Dalam praktik nyata, hampir semua proyek profesional menggunakan cara eksternal karena kemudahannya dalam pengelolaan dan efisiensinya.

3.1.2. Silahkan buat 1 file html yang isinya menampilkan output dari javascript yang dibuat menggunakan Fungsi console.log(), Fungsi alert(), Fungsi document.write(), dan innerHTML. (Isi teks html dan javascript bisa anda variasikan sendiri).

Jawab:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Empat Cara SIGMA Menampilkan Output JavaS-
cript</title>
</head>
<body>

  <h1>Empat Cara SIGMA Menampilkan Output JavaS-
cript</h1>
  <hr>

  <h2>1. console.log()</h2>
  <p>Buka Console di browser (F12) untuk melihat out-
putnya.</p>
  <button onclick="console.log('Halo dari con-
sole.log!')">Jalankan console.log()</button>

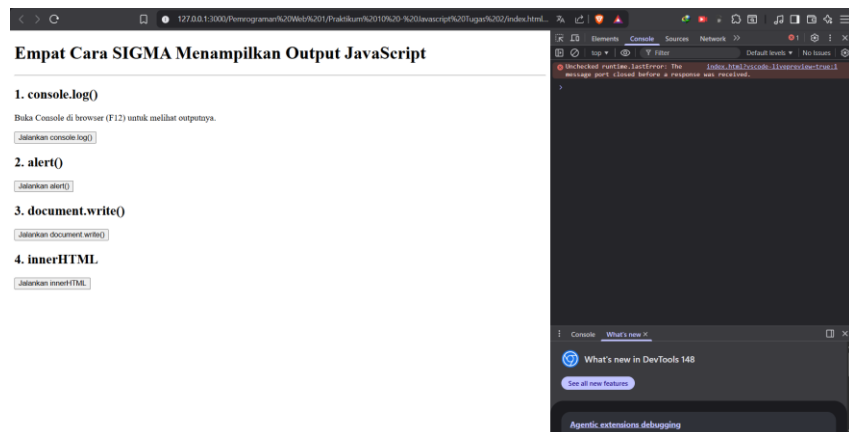
  <h2>2. alert()</h2>
  <button onclick="alert('Halo dari alert!')">Jalankan
alert()</button>

  <h2>3. document.write()</h2>
  <button onclick="document.write('Halo dari docu-
ment.write!')">Jalankan document.write()</button>

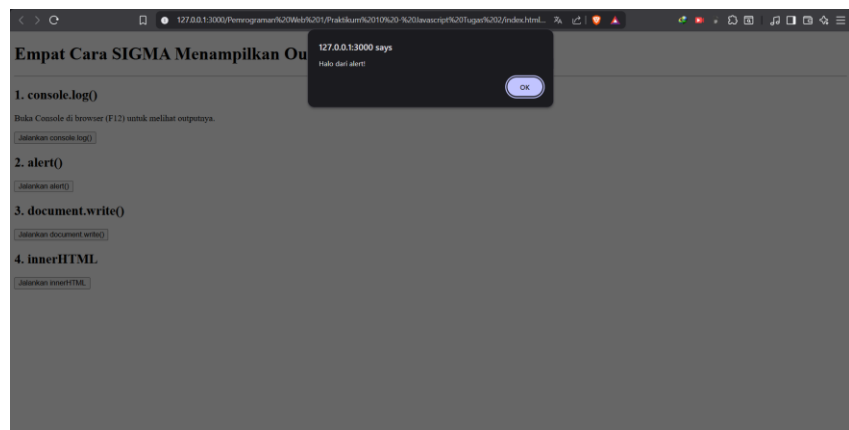
  <h2>4. innerHTML</h2>
  <button on-
click="document.getElementById('output').innerHTML =
'Halo dari innerHTML!'">Jalankan innerHTML</button>
  <p id="output"></p>
```

```
</body>
</html>
```

Dengan outputnya sebagai berikut:



Gambar 3.5 Tampilan output console.log()



Gambar 3.6 Tampilan output alert()



Gambar 3.7 Tampilan output document.write()

Empat Cara SIGMA Menampilkan Output JavaScript

1. console.log()

Buka Console di browser (F12) untuk melihat outputnya.

2. alert()

3. document.write()

4. innerHTML

Halo dari innerHTML!

Gambar 3.8 Tampilan output innerHTML

JavaScript menyediakan beberapa cara untuk menampilkan output, masing-masing dengan tujuan dan perilaku yang berbeda. Empat cara yang paling umum digunakan adalah console.log(), alert(), document.write(), dan innerHTML. Memahami perbedaan keempatnya penting agar bisa memilih cara yang tepat sesuai kebutuhan.

console.log() adalah cara menampilkan output yang

paling sering digunakan oleh para developer, namun outputnya tidak terlihat langsung di halaman web. Pesan yang dikirim melalui `console.log()` hanya muncul di Console browser, yang bisa dibuka dengan menekan tombol F12 lalu memilih tab Console. Pada contoh di file `index.html`, ketika tombol diklik, perintah `console.log('Halo dari console.log!')` dijalankan dan teks tersebut muncul di Console. Cara ini sangat berguna untuk keperluan debugging, yaitu mengecek nilai variabel atau memastikan suatu bagian kode sudah berjalan, tanpa mengganggu tampilan halaman yang dilihat pengguna.

`alert()` adalah cara menampilkan output berupa kotak dialog pop-up yang muncul langsung di atas halaman browser. Ketika `alert('Halo dari alert!')` dijalankan, browser akan menghentikan sementara semua aktivitas di halaman dan menampilkan kotak kecil berisi pesan tersebut beserta tombol OK. Pengguna harus menekan OK terlebih dahulu sebelum bisa kembali berinteraksi dengan halaman. Cara ini efektif untuk memberikan peringatan atau informasi penting yang harus dilihat pengguna, namun penggunaannya perlu dibatasi karena bisa terasa mengganggu jika terlalu sering muncul.

`document.write()` adalah cara menampilkan output dengan langsung menuliskan teks atau konten HTML ke dalam halaman web. Pada contoh file `index.html`, perintah `document.write('Halo dari document.write!')` akan menuliskan teks tersebut ke halaman. Namun ada perilaku khusus yang perlu diperhatikan: jika `document.write()` dipanggil setelah halaman selesai dimuat sepenuhnya, seluruh isi halaman yang sudah ada akan terhapus dan digantikan hanya dengan teks yang baru di-

tulis tersebut. Itulah mengapa cara ini jarang digunakan dalam proyek nyata dan lebih sering hanya muncul di contoh-contoh pembelajaran dasar.

`innerHTML` adalah cara menampilkan output dengan mengubah atau mengisi konten di dalam elemen HTML tertentu yang sudah ada di halaman. Pada contoh file `index.html`, perintah `document.getElementById('output').innerHTML = 'Halo dari innerHTML!'` bekerja dengan cara pertama mencari elemen yang memiliki `id="output"`, lalu mengisi bagian dalamnya dengan teks tersebut. Berbeda dengan `document.write()` yang bisa menghapus seluruh halaman, `innerHTML` hanya menyentuh elemen yang dituju saja, sehingga jauh lebih aman dan terkontrol. Cara ini adalah yang paling banyak digunakan dalam pengembangan web modern karena fleksibilitasnya — bahkan bisa digunakan untuk menyisipkan elemen HTML baru, bukan hanya teks biasa.

Secara keseluruhan, `console.log()` dipakai untuk kebutuhan debugging oleh developer, `alert()` dipakai untuk menampilkan pesan penting kepada pengguna, `document.write()` umumnya hanya dipakai untuk pembelajaran dasar karena sifatnya yang merusak konten halaman, dan `innerHTML` adalah pilihan terbaik dan paling fleksibel untuk menampilkan atau memperbarui konten di halaman web secara dinamis.

4. Kesimpulan

JavaScript merupakan bahasa pemrograman yang sangat penting dalam pengembangan web modern karena mampu menciptakan halaman web yang interaktif, dinamis, dan responsif. JavaScript tidak hanya digunakan pada sisi *client-side* di browser, tetapi juga dapat diterapkan pada sisi server melalui teknologi seperti Node.js. Dengan dukungan DOM, *event handling*, dan Fetch API, JavaScript memungkinkan pengembang untuk memanipulasi tampilan halaman, merespons interaksi pengguna, serta melakukan komunikasi data secara asinkron tanpa memuat ulang halaman.

Selain itu, JavaScript memiliki sintaks yang relatif mudah dipahami dan mendukung berbagai paradigma pemrograman sehingga banyak digunakan dalam berbagai platform pengembangan aplikasi. Contoh implementasi sederhana seperti manipulasi DOM, penanganan *event*, dan penggunaan Fetch API menunjukkan bahwa JavaScript memiliki peran besar dalam meningkatkan pengalaman pengguna pada website modern. Oleh karena itu, pemahaman dasar mengenai JavaScript sangat penting bagi pengembang web dalam membangun aplikasi yang efektif, interaktif, dan efisien.

5. Daftar Pustaka

- [1] MDN Web Docs, “JavaScript Language Overview,” Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Language_overview. [Accessed: Mei 2026].
- [2] A. Wirfs-Brock dan B. Eich, “JavaScript: The First 20 Years,” *Proceedings of the ACM on Programming Languages*, vol. 4, no. HOPL, 2020.
- [3] MDN Web Docs, “Document Object Model (DOM),” Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model. [Accessed: Mei 2026].
- [4] MDN Web Docs, “Using the Fetch API,” Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch. [Accessed: Mei 2026].
- [5] MDN Web Docs, “Cross-site scripting (XSS),” Mozilla. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>. [Accessed: Mei 2026].
- [6] MDN Web Docs, “Web Performance Best Practices,” Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn/Performance/Best_practices. [Accessed: Mei 2026].
- [7] MDN Web Docs, “JavaScript Technologies Overview,” Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/JavaScript_technologies_overview. [Accessed: Mei 2026]
- [8] <https://github.com/Hudaws/PPW-BAB-10-11>